NFC-Based Attendance System Using Flutter and Google Sheets

Report of 3rd March:

1. Abstract

This project presents the development of an NFC-based attendance system using Flutter for the frontend and Google Apps Script with Google Sheets as the backend. The system allows users to simulate NFC-based attendance marking, where data is transmitted from a Flutter application to a Google Sheets database via a web API. The system was designed to automate attendance tracking efficiently and minimize manual errors. Various issues encountered during development, including API configuration, HTTP communication, and data integration, were resolved to achieve a fully functional system.

2. Introduction

Traditional attendance systems are often time-consuming and prone to human error. With the advancement of mobile and web technologies, automated systems using NFC (Near Field Communication) can significantly improve efficiency and accuracy.

This project aims to develop a **hybrid NFC-based attendance system** where:

- Flutter is used to build the user interface,
- Google Apps Script acts as a backend API,
- Google Sheets serves as the database.

The system simulates NFC scanning and records attendance data in real time.

3. Objectives

- To design a Flutter-based application for attendance marking
- To integrate Flutter with Google Apps Script via HTTP requests
- To store attendance data in Google Sheets
- To implement a reliable and error-free communication system
- To analyze and resolve real-world API and integration issues

4. System Architecture

4.1 Components

- **Frontend:** Flutter (Windows/Desktop application)
- **Backend:** Google Apps Script (Web API)
- **Database:** Google Sheets
- **Communication Protocol:** HTTP (GET/POST requests)

4.2 Workflow

1. User initiates attendance by pressing a button (simulated NFC scan)
2. Flutter application sends an HTTP request to the Apps Script API
3. Apps Script processes the request and extracts parameters
4. Data is appended to Google Sheets
5. A response is sent back to the Flutter application
6. The UI updates with the result

5. System Implementation

5.1 Flutter Application

The Flutter application provides a simple user interface with:

- A status display message
- A button to simulate NFC scanning

The application sends HTTP requests using the http package and updates the UI based on the response.

5.2 Google Apps Script Backend

The backend is implemented using Google Apps Script, which:

- Accepts HTTP requests
- Extracts parameters such as student ID
- Connects to a specific Google Sheet

- Appends attendance data
- Returns a response to the client

5.3 Google Sheets Database

The system uses Google Sheets as a lightweight database. The sheet includes fields such as:

- Date
- Entry Time
- Exit Time
- Student ID
- Room
- Type (IN/OUT)
- Method (e.g., SIM)
- Total Joined Class

6. Challenges and Debugging

During development, several issues were encountered:

6.1 Command and Setup Errors

Initial issues included incorrect terminal commands, which prevented dependency installation. These were resolved by correcting command syntax.

6.2 HTTP Method Mismatch

The application initially faced HTTP 405 errors due to mismatched request methods. The backend expected a different HTTP method than what the frontend sent. This was resolved by aligning both sides to use the same method.

6.3 Missing API Functions

The backend initially lacked the required request handler, causing errors when accessing the API. Adding the appropriate handler function resolved this issue.

6.4 HTML Response Issues

Instead of returning clean data, the API initially returned HTML content. This caused improper display in the Flutter UI. The issue was resolved by formatting responses as plain text using the correct content service.

6.5 Redirect Handling (302 Error)

Google Apps Script introduced redirect responses, which were not handled properly by the client. Adjusting the request handling and using the correct endpoint resolved the issue.

6.6 Spreadsheet Integration Errors

Data was not initially stored due to mismatches in sheet naming and configuration. Since Google Sheets is case-sensitive, correcting the sheet name ensured successful data insertion.

7. Final System Behavior

After resolving all issues, the system achieved the following:

- Successful communication between Flutter and Apps Script
- Real-time data insertion into Google Sheets
- Proper UI feedback in the application
- Stable and consistent performance

The system now allows multiple attendance entries and correctly records both entry and exit data.

8. Results

The final implementation demonstrates:

- Accurate data transmission
- Reliable backend processing
- Correct storage in Google Sheets

- User-friendly interface

The system operates without runtime errors and successfully logs attendance records.

9. Conclusion

The NFC-based attendance system was successfully developed and implemented using Flutter and Google Apps Script. The project demonstrates how modern web technologies can be integrated to create efficient, automated solutions.

Through systematic debugging and problem-solving, all technical challenges were resolved, resulting in a fully functional system. This project highlights the importance of proper API design, consistent communication protocols, and accurate data handling.

10. Future Enhancements

- Integration with real NFC hardware devices
- User authentication and security features
- Cloud database integration (e.g., Firebase)
- Admin dashboard for monitoring attendance
- Data analytics and reporting features